Prog II:

Tipos Estruturados

Profª. Tainá Isabela

Estrutura

Estrutura é um tipo defi nido pelo usuário, em que sua declaração serve para defi nir as propriedades dos dados desse novo tipo. Uma estrutura é um conjunto de variáveis simples.

As variáveis em uma estrutura podem ser de diferentes tipos: podem ser int, float, e assim por diante.

Estrutura

Na programação C++, estruturas são um dos elementos básicos para entender objetos e classes. Tipicamente, uma estrutura é uma coleção de dados, enquanto uma classe é uma coleção de dados e funções.

Estrutura

A linguagem C++ provê suporte para estruturas, as quais podem ser utilizadas quando se necessita criar um tipo de dado novo.

Note que se trata de um tipo de dado defi nido pelo usuário ou programador, geralmente citado como user-defi ned data type.

Esse tipo de dado novo que se pretende defi nir agrupa, de modo lógico, vários campos ou membros que fazem parte da estrutura que será criada.

Estrutura

```
struct ponto {
double x;
double y;
};

struct circulo {
ponto centro;
double raio;
double área; };

 ponto p1, p2;
```

Estrutura

A linguagem C++ provê suporte para estruturas, as quais podem ser utilizadas quando se necessita criar um tipo de dado novo.

Note que se trata de um tipo de dado defi nido pelo usuário ou programador, geralmente citado como user-defi ned data type.

Esse tipo de dado novo que se pretende defi nir agrupa, de modo lógico, vários campos ou membros que fazem parte da estrutura que será criada.

Definição de estrutura

A especifi cação de uma estrutura diz como a estrutura está organizada, ou seja, ela define os membros que a estrutura possui:

.

```
struct data {     // especifica a estrutura
int dia;          // dia da data
int mes;          // mes da data
int ano;          // ano da data
};
```

Acesso a Membros de Estrutura

Após a variável de uma estrutura ter sido defi nida, seus membros podem ser acessados usando um operador do tipo ponto (ou dot operator).

```
hoje.dia
hoje.mes
hoje.ano
```

Note que o membro da estrutura é escrito em três partes: o nome da variável da estrutura (hoje), o operador ponto (.) e o nome do membro, por exemplo, no primeiro membro (dia).

Acesso a Membros de Estrutura

```
#include <iostream>
using namespace std;
// Programa para ilustrar a inicializacao de estrutura
struct data {
  int dia;
  int mes;
  int ano;
};
int main()
{
  data hoje = {25, 12, 2009};
  data novaData;
  cout << "\nData de hoje informada: " << hoje.dia << " / "
  << hoje.mes << " / " << hoje.ano << endl;
  novaData = hoje;
  cout << "\nNova data informada: " << hoje.dia << " / " <<
  hoje.mes << " / " << hoje.ano << endl << endl;
  system("PAUSE");
  Return 0;
}
```

Especificador enum

O especificador ***enum*** define o conjunto de todos os nomes que serão os valores possíveis desse tipo.

Tais valores são chamados de membros. O tipo enum diasDaSemana tem sete membros (Seg, Ter,..., Sab). Enumerado vem do fato de que todos os valores são listados. Usando enum, você está atribuindo um nome específico para cada valor possível.

Estruturas e Classes

Estruturas em C++ foram inicialmente utilizadas com o objetivo de agrupar vários membros de dados para realizar determinada funcionalidade, conforme visto anteriormente no capítulo.

Por outro lado, uma classe é uma coleção de objetos que possuem as mesmas propriedades. Em outras palavras, classe é o conjunto de objetos que possuem o mesmo comportamento.

Uma classe serve como um modelo ou template para defi nir as características dos objetos. O termo classe é, na realidade, uma abreviação do termo classe de objetos.

Estruturas e Classes

Diferenças entre estruturas e classes compreendem a defi nição de uma estrutura, na qual você utiliza a palavra reservada struct, enquanto na definição de uma classe usa a palavra class.

Além disso, todos os membros de uma estrutura são de natureza pública (por default). Já em uma classe, todos os membros são privados.

```
struct exemplo {
int a;
int b;
float c;
};
```

```
class exemplo {
private int a;
private int b;
private int c;
};
```

Funções

Um dos principais princípios para o desenvolvimento de sistemas, e mais especificamente de software, compreende a decomposição estrutural.

Nesse sentido, a função é entidade utilizada para organizar o software em unidades (ou funções). Utilizar função para organizar o software é a base da programação estruturada.

Uma função agrupa muitas instruções em uma unidade e lhe atribui um nome. Cabe destacar que essa unidade pode ser chamada de outras partes do programa. Além disso, qualquer sequência de instruções que apareça em um programa mais de uma vez é candidata para se tornar uma função.

```
#include <iostream>;
tipoRetorno nomeFuncao (arg1, arg2,...) {
...; // corpo da função
}
int main()
{...; // corpo da funcao principal main() }
```

- tipoRetorno é o tipo de dado retornado pela função;
- nomeFuncao é o nome atribuído à função;
- arg1, arg2,... são os argumentos (ou parâmetros) passados à função quando ela é chamada; cada argumento deve ser precedido pelo seu respectivo tipo;
- corpo da funcao compreende às instruções da função.

Funções Recursivas

Funções recursivas são funções que obtêm um resultado através de várias chamadas à própria função.

Entretanto, as chamadas recursivas devem ser limitadas para evitar o uso excessivo de memória. Nesse sentido, deve-se estar atento para que a função recursiva verifique a condição de término de uma recursão.

Retorno de Valores de Funções

Quando uma função termina sua execução, ela pode retornar um valor para o programa chamador. Geralmente, esse valor retornado é uma resposta para o problema resolvido pela função.

É necessário sempre incluir o tipo de retorno de uma função na declaração da função. Se, por acaso, a função não precisa retornar valor, então use void para indicar isso.

Do contrário, o compilador assumirá que a função retorna um valor do tipo int. É recomendável sempre definir o tipo de dado de retorno, mesmo que ele seja um do tipo int (inteiro). Adicionalmente, é recomenável usar parênteses ao redor da expressão de return (valor a retornar).

Passagem de Argumento por Referência

Passar argumentos por referência, contudo, utiliza um mecanismo diferente. Em vez de um valor ser passado para a função, uma referência à variável original é passada. Na realidade, o endereço da variável é passado.

A principal vantagem de passar argumento por referência é que a função pode ter acesso às variáveis no programa chamador. Um dos benefícios é que a função pode retornar mais de um valor para o programa chamador.

```
void obtemIntFracao(double n, double& nInt, double& nFracao)
...
void obtemIntFracao(double, double&, double&)
```

Funções Sobrecarregadas

Uma função sobrecarregada executa diferentes atividades dependendo do(s) tipo(s) de dado(s) passado(s) a ela. Considere a função calculaQuadrado(), que deve retornar o quadrado de um valor n que você tenha digitado.

Em tal situação, você pode digitar um número inteiro ou real e o programa deve calcular o quadrado do valor a depender do tipo está se passando.

```
int calculaQuadrado(int x) { … }

double calculaQuadrado(double x) { … .}
```

Funções Inline

Usar funções adequadamente em um programa é essencial. As funções economizam espaço de memória porque todas as chamadas à função fazem com que o mesmo fragmento de código seja executado. Assim, o corpo da função não precisa ser duplicado na memória.

Embora essa sequência de eventos possa economizar memória, ela consome mais tempo. Deve haver uma instrução para o salto à função, instruções para salvar registros, instruções para colocar argumentos na pilha no programa chamador e removê-los da pilha na função (se houver argumentos), instruções para restaurar os registros e uma instrução para retornar ao programa chamador.

Funções Inline

Objetivando economizar tempo de execução em funções pequenas, podemos colocar o código do corpo da função junto com o programa chamador. Ou seja, toda vez que houver uma chamada à função no código-fonte, o código da função é inserido em vez de fazer um salto para a função.

Perceba que, se uma função é muito pequena, as instruções necessárias para chamá-la podem consumir também espaço de memória e haverá consumo de tanto tempo quanto de espaço em memória.

Funções Inline

A solução para isso é usar função inline. Esse tipo de função é escrito como uma função normal no arquivo-fonte, mas compila no código inline em vez da função. O arquivo fi ca bem organizado e fácil de ler, desde que a função seja mostrada como uma entidade separada.

Entretanto, quando o programa é compilado, o corpo da função é inserido no programa em todos os locais onde houver uma chamada de função. As funções que são pequenas (com uma ou duas instruções, por exemplo) são candidatas a se tornarem inline.

```
inline tipoRetorno nomeFuncao(listaParametros)
```

Argumentos Default

Uma função pode ser chamada sem ter todos os seus argumentos especifi cados. Para isso acontecer, a declaração de função deve fornecer os valores default para aqueles argumentos não especificados.

```
void imprimeCaracteres(char='#', int = 50);
```

	automatic	static auto	external
Objetivo	Variáveis usadas por uma única função	Idêntico a auto, porém deve reter o valor quando a função termina a execução	Variáveis usadas por várias funções
Valor inicial	Não inicializada	0	0
Tempo de vida	Função	Programa	Programa
Visibilidade	Função	Função	Arquivo

Argumentos Default

Uma função pode ser chamada sem ter todos os seus argumentos especifi cados. Para isso acontecer, a declaração de função deve fornecer os valores default para aqueles argumentos não especificados.

```
void imprimeCaracteres(char='#', int = 50);
```

Arrays

Um array em C++ é uma estrutura de dados que permite armazenar múltiplos valores do mesmo tipo em uma única variável, acedendo cada elemento por um índice numérico. Arrays são úteis para organizar e manipular coleções de dados de forma eficiente.

```
int numeros[5]; // Array de 5 inteiros
float notas[10]; // Array de 10 floats
char nome[100]; // Array de 100 caracteres

int valores[3] = {10, 20, 30}; // Array de 3 inteiros com valores definidos
```

Arrays

Os índices começam em 0. Para aceder ou modificar elementos:

.

```
numeros[0] = 7; // Define o primeiro elemento como 7
notas[2] = 9.5; // Altera o terceiro elemento para 9.5
```

Arrays Multidimensionais (Matrizes)

Arrays podem ter mais de uma dimensão, como matrizes (arrays 2D):

.

```
int matriz[3][4] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12}
    };

    for (int i = 0; i < 3; ++i) {
        for (int j = 0; j < 4; ++j) {
            cout << matriz[i][j] << " ";
        }
        cout << endl;
    }
```

Arrays Multidimensionais (Matrizes)

- O tamanho do array deve ser conhecido em tempo de compilação, a menos que use arrays dinâmicos.
- O índice do array começa em 0.
- Acessar índices fora do intervalo causa comportamento indefinido.
- Arrays são fundamentais para manipular conjuntos de dados em C++. Eles são a base para estruturas mais avançadas e para algoritmos eficientes.

Arrays dinâmicos

Um array dinâmico em C++ é uma estrutura de dados cujo tamanho pode ser definido em tempo de execução, ao contrário dos arrays estáticos, que têm tamanho fixo determinado na compilação.

Arrays dinâmicos são úteis quando não se sabe previamente quantos elementos serão necessários ou quando o tamanho do array pode variar durante a execução do programa.

Para criar um array dinâmico, utiliza-se a palavra-chave new, que aloca memória dinamicamente e retorna um ponteiro para o primeiro elemento do array. Após usar o array, é fundamental libertar a memória com **delete**[] para evitar vazamentos

Arrays dinâmicos

```
tipo *nomeArray = new tipo[tamanho];
```

- **tipo**: tipo de dado do array (ex: int, float, char)
- **nomeArray**: nome do ponteiro que referencia o array
- **tamanho**: número de elementos, definido em tempo de execução

```
int main() {
    int n;
    cout << "Digite o tamanho do array: ";
    cin >> n;

    int *array = new int[n]; // Criação do array dinâmico

    // Preenchendo o array
    for (int i = 0; i < n; i++) {
        cout << "Digite o elemento " << i << ": ";
        cin >> array[i];
    }

    // Exibir os elementos
    cout << "Elementos do array: ";
    for (int i = 0; i < n; i++) {
        cout << array[i] << " ";
    }
    cout << endl;

    delete[] array; // Libertação da memória
    return 0;
}
```

Array dinâmico de caracteres (string dinâmica)

```
#include <iostream>
#include <cstring>
using namespace std;

int main() {
    int tamanho = 20;
    char *nome = new char[tamanho];

    strcpy(nome, "Exemplo de string");
    cout << nome << endl;

    delete[] nome;
    return 0;
}
```

Em resumo:

OPERAÇÃO	SINTAXE/EXEMPLO
Criar array dinâmico	`int *arr = new int[tamanho];`
Acessar elemento	`arr[i]`
Inicializar com zero	`int *arr = new int[tamanho]();`
Libertar memória	`delete[] arr;`

Adicionar/Remover elementos

Em C++, arrays dinâmicos criados com **new** têm tamanho fixo após a alocação: não existe um mecanismo nativo para adicionar ou eliminar elementos diretamente como em um std::vector.

Para modificar o tamanho de um array dinâmico, é necessário realocar a memória manualmente, copiando os dados para um novo array de tamanho ajustado.

Adicionar um elemento a um array dinâmico

1. Alocar um novo array com tamanho maior;
2. Copiar os elementos do array antigo para o novo;
3. Adicionar o novo elemento;
4. Libertar a memória do array antigo;
5. Atualizar o ponteiro;

```cpp
#include <iostream>
using namespace std;

int main() {
    int n = 3;
    int* arr = new int[n]{1, 2, 3};

    // Adicionar um novo elemento (por exemplo, 4)
    int novoTamanho = n + 1;
    int* novoArr = new int[novoTamanho];

    // Copiar elementos antigos
    for (int i = 0; i < n; i++) {
        novoArr[i] = arr[i];
    }
    // Adicionar novo elemento
    novoArr[n] = 4;

    delete[] arr; // Libertar memória antiga
    arr = novoArr;
    n = novoTamanho;

    // Exibir resultado
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    delete[] arr; // Libertar memória final
    return 0;
}
```

Eliminar um elemento de um array dinâmico

1. Alocar um novo array com tamanho menor;
2. Copiar todos os elementos, exceto o que será removido;
3. Libertar a memória do array antigo;
4. Atualizar o ponteiro;

```
#include <iostream>
using namespace std;

int main() {
    int n = 4;
    int* arr = new int[n]{1, 2, 3, 4};

    int indiceRemover = 1; // Remove o elemento na posição 1 (valor 2)
    int novoTamanho = n - 1;
    int* novoArr = new int[novoTamanho];

    for (int i = 0, j = 0; i < n; i++) {
        if (i != indiceRemover) {
            novoArr[j++] = arr[i];
        }
    }

    delete[] arr;
    arr = novoArr;
    n = novoTamanho;

    // Exibir resultado
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    delete[] arr;
    return 0;
}
```

Observações importantes

- Arrays dinâmicos não redimensionam automaticamente. O processo envolve sempre criar um novo array, copiar e libertar a memória antiga.
- Para operações frequentes de inserção/remoção, prefira std::vector, que já gere isso internamente e de forma eficiente.
- Liberte sempre a memória antiga com delete[] para evitar vazamentos de memória.

Questionário

Exercícios

1. Escreva um programa que solicite do usuário dia, mês e ano atual e, em seguida, o programa exibe a data informada.
2. Escreva um programa que solicite ao usuário digitar dois valores inteiros e, em seguida, o programa principal chama uma função, denominada soma, passando os parâmetros que o usuário digitou.
3. Escreva um programa que solicite ao usuário digitar uma quan a em dólares que você deseja que seja conver da para reais. Seu programa deve implementar uma função inline. Após a entrada da quan a em dólares, o programa calcula e retorna a quantia convertida em reais.
4. Elabore um programa que cria um array bidimensional com valores do tipo double. Chame esse array de despesas[ANO][TRIMESTRE]. Adicionalmente, você deve definir ANO=2 e TRIMESTRE=4. Em seguida, deve criar um laço que solicite ao usuário digitar os valores de despesas para cada um dos semestres nos dois anos. Ao final, o programa exibirá uma tabela que mostra os valores das despesas de cada ano, sendo listadas por trimestre, além do valor total de despesas nos dois anos